Alter column position

From PostgreSQL wiki

Languages: **English** • 日本語 • Русский

Many people new to postgresql often ask if it has support for altering column positions within a table. Currently it does not; if you want to change column positions, you must either recreate the table, or add new columns and move data. The idea of allowing re-ordering of column position is not one the postgresql developers are against, it is more a case where no one has stepped forward to do the work. The rest of this document is meant to explain the workarounds, and track the issues that must be dealt with should someone want to implement this functionality.

There are two main reasons why being able to alter column position would be useful within postgres

1. physical layout can be optimized by putting fixed size columns at the start of the table
2. ordering columns can make working with a table easier, either by putting result sets in an order that is visually appealing, or by grouping columns based on similar function within a table.

Postgres currently defines column order based on the attnum column of the pg_attribute table. The only way to change column order is either by recreating the table, or by adding columns and rotating data until you reach the desired layout.

Contents

- 1 Alter column workarounds
  - 1.1 Recreate the table
  - 1.2 Add columns and move data
  - 1.3 Hide the differences with a view
- 2 Adding alter column syntax into postgres

Alter column workarounds

Recreate the table

In our first example, we have a table named film_actor, which contains three columns, an actor_id, a film_id, and a last_update column. We have noticed that many developers often confuse which id is the films and which is the actors, so to eliminate this, we want to change the order of the columns.

```
pagila=# \d film_actor
                    Table "public.film_actor"
   Column    |            Type             |       Modifiers
-------------+-----------------------------+------------------------
 actor_id    | smallint                    | not null
 film_id     | smallint                    | not null
 last_update | timestamp without time zone | not null default now()
Indexes:
```

```
    "film_actor_pkey" PRIMARY KEY, btree (actor_id, film_id)
    "idx_fk_film_id" btree (film_id)
Foreign-key constraints:
    "film_actor_actor_id_fkey" FOREIGN KEY (actor_id) REFERENCES actor(actor_id) ON UPDATE CASCADE ON DELETE RESTRICT
    "film_actor_film_id_fkey" FOREIGN KEY (film_id) REFERENCES film(film_id) ON UPDATE CASCADE ON DELETE RESTRICT
Triggers:
    last_updated BEFORE UPDATE ON film_actor FOR EACH ROW EXECUTE PROCEDURE last_updated()
```

To do this, we will create a new table with our desired column order, update it with the data, then drop the old table, resolving any dependencies.

```
pagila=# begin;
BEGIN
pagila=# create table film_actor_new (
pagila-# film_id smallint references film,
pagila-# actor_id smallint references actor,
pagila-# last_update timestamp not null default now(),
pagila-# primary key (actor_id,film_id));
NOTICE:  CREATE TABLE / PRIMARY KEY will create implicit
index "film_actor_new_pkey" for table "film_actor_new"
CREATE TABLE
pagila=# insert into film_actor_new select film_id, actor_id, last_update from film_actor;
INSERT 0 5462
pagila=# drop table film_actor cascade;
NOTICE:  drop cascades to rule _RETURN on view nicer_but_slower_film_list
NOTICE:  drop cascades to view nicer_but_slower_film_list
NOTICE:  drop cascades to rule _RETURN on view film_list
NOTICE:  drop cascades to view film_list
NOTICE:  drop cascades to rule _RETURN on view actor_info
NOTICE:  drop cascades to view actor_info
DROP TABLE
pagila=# alter table film_actor_new RENAME to film_actor;
ALTER TABLE
pagila=# \d film_actor
                     Table "public.film_actor"
   Column    |            Type             |       Modifiers
-------------+-----------------------------+------------------------
 film_id     | smallint                    | not null
 actor_id    | smallint                    | not null
 last_update | timestamp without time zone | not null default now()
Indexes:
    "film_actor_new_pkey" PRIMARY KEY, btree (actor_id, film_id)
Foreign-key constraints:
    "film_actor_new_actor_id_fkey" FOREIGN KEY (actor_id) REFERENCES actor(actor_id)
    "film_actor_new_film_id_fkey" FOREIGN KEY (film_id) REFERENCES film(film_id)
```

If we were to continue, we would need to recreate the views referenced above to reference our new version of the table. Also don't forget, we would also need to recreate the missing indexes on our new table, and recreate the trigger, and then rename all of the tables / indexes / etc... to ensure our final schema matches the previous schema. While you can do all of this work in a transaction, it is important to be thorough about the work, as you can run into a lot of dependency issues.

Add columns and move data

Sometimes it is too much trouble to recreate a table, so instead you can use the add column and move data approach. In this example, we have an actor with id, first and last name column, and alast_updated column. We are going to be adding a middle_name column into our table, and we would like to add it between the first_name and last_name columns.

```
pagila=# \d actor
                                      Table "public.actor"
   Column    |            Type             |                         Modifiers
-------------+-----------------------------+-----------------------------------------------------------
 actor_id    | integer                     | not null default nextval('actor_actor_id_seq'::regclass)
 first_name  | character varying(45)       | not null
 last_name   | character varying(45)       | not null
 last_update | timestamp without time zone | not null default now()
Indexes:
    "actor_pkey" PRIMARY KEY, btree (actor_id)
    "idx_actor_last_name" btree (last_name)
Triggers:
    last_updated BEFORE UPDATE ON actor FOR EACH ROW EXECUTE PROCEDURE last_updated()
```

To do this, we will first add the middle name column (which will go onto the end), then add a new last_name column, and another new last_update column, update the respective columns, and then drop the old columns.

```
pagila=# begin;
BEGIN
pagila=# alter table actor add column middle_name text, add column lname text, add column lupdate text;
ALTER TABLE
pagila=# update actor set lname=last_name, lupdate=last_update;
UPDATE 200
pagila=# alter table actor drop column last_name cascade, drop column last_update cascade;
NOTICE:  drop cascades to rule _RETURN on view nicer_but_slower_film_list
NOTICE:  drop cascades to view nicer_but_slower_film_list
NOTICE:  drop cascades to rule _RETURN on view film_list
NOTICE:  drop cascades to view film_list
NOTICE:  drop cascades to rule _RETURN on view actor_info
NOTICE:  drop cascades to view actor_info
ALTER TABLE
pagila=# alter table actor rename column lname to last_name;
ALTER TABLE
pagila=# alter table actor  rename column lupdate to last_update;
ALTER TABLE
pagila=# \d actor
                                     Table "public.actor"
   Column    |         Type          |                         Modifiers
-------------+-----------------------+-----------------------------------------------------------
 actor_id    | integer               | not null default nextval('actor_actor_id_seq'::regclass)
 first_name  | character varying(45) | not null
 middle_name | text                  |
 last_name   | text                  |
 last_update | text                  |
Indexes:
    "actor_pkey" PRIMARY KEY, btree (actor_id)
Triggers:
    last_updated BEFORE UPDATE ON actor FOR EACH ROW EXECUTE PROCEDURE last_updated()
```

Again, before we completed this change, we would need to recreate the above views, and deal with any indexes, triggers, or other dependencies that might arise. It is also important that the update will cause all rows in the table to be rewritten, which will then require maintenance (most likely some for of cluster/reindex based on your table)

Hide the differences with a view

for completeness, there is another method for changing table order, which is to hide the changes with a view. The basic idea is you add any new columns (if you're adding a new column), rename the table, and create a view with the old table name that contains your desired column order. As of PostgreSQL 9.3, this view should act just like an ordinary table with regard to writes. While this works for logical column re-ordering, if you want to re-order to optimize physical layout, you would still need to use one of the above reasons.

Adding alter column syntax into postgres

Since the above methods have a number of issues, it has often been expressed that we would like to add capabilities for postgres to allow reordering of columns. The flip side of this is that it would also be desirable for postgres to automatically order columns physically for optimum layout, regardless of the logical order they are given in. The current problem with implementing this lies in that currently postgres uses the same identifiers for both the logical and physical position within a table. The current hot plan for solving this would be to change the system to reference three identifiers... a permanent identifier for the column, as well as a separate logical and physical identifier. This would allow places that need to deal specifically with column order at the logical level (ie. select *) to reference the logical number, while places that interact with disk system can access the physical number, and all other places just use the column's permanent id.

- The above plan is taken from the following email http://archives.postgresql.org/pgsql-hackers/2006-12/msg00983.php.
- The TODO list references this email, http://archives.postgresql.org/pgsql-hackers/2006-12/msg00782.php, which is a fun read if you want more back story.

The other bit is that we would also need to determine syntax for how users would interact with this new functionality. The most common implementations typically use a set of BEFORE/AFTER keywords, followed by an existing column name.

Retrieved from "https://wiki.postgresql.org/index.php?title=Alter_column_position&oldid=23469"
Category: Administration

- This page was last modified on 20 October 2014, at 03:14.